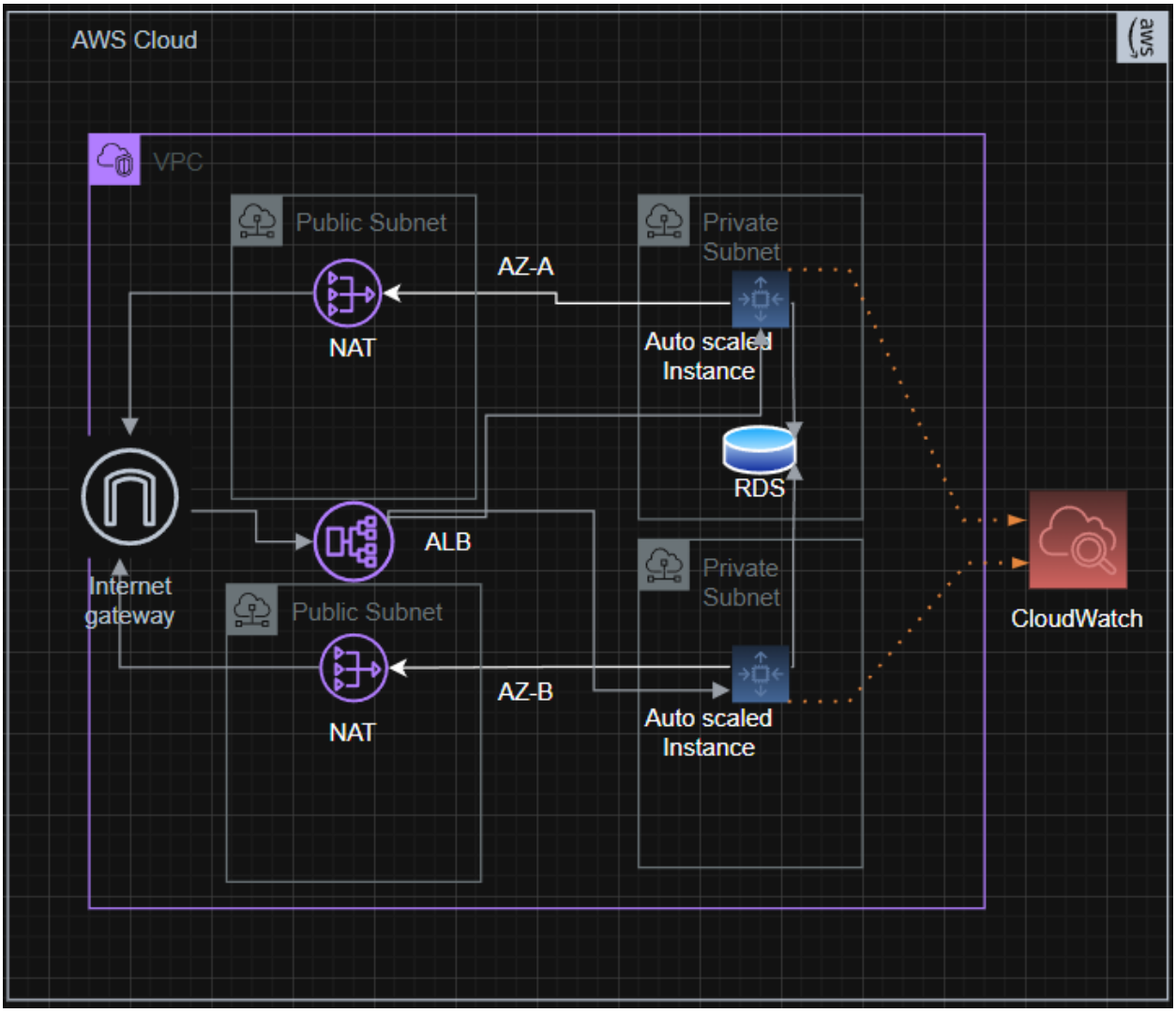
Cloud Architecture

Assignment-1 (Report)

Submitted By: Fahamidul Hasan

Contents

Architecture Diagram



- **VPC:** Contains two public and two private subnets in different availability zones to provide higher availability.

- **IGW:** Attached to the VPC to allow both inbound and outbound traffic.

- **ALB:** Load Balancer is placed in public subnets, and it receives inbound traffic from IGW and efficiently directs them to the auto scaled instances in private subnets. It also prevents direct access to the private subnets.

- **NAT:** Placed in the public subnets, it receives outbound requests from auto scaled instances in the private subnets and directs them to the IGW. It prevents the

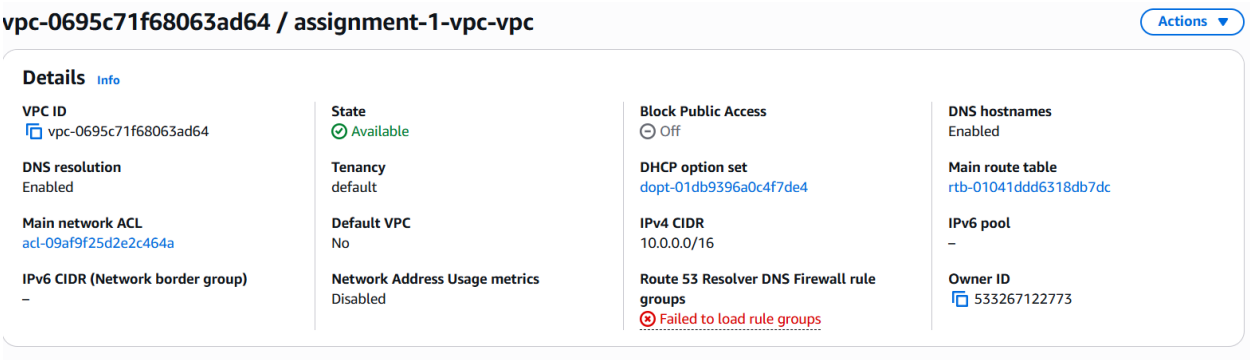
instances in private subnet of becoming exposed.

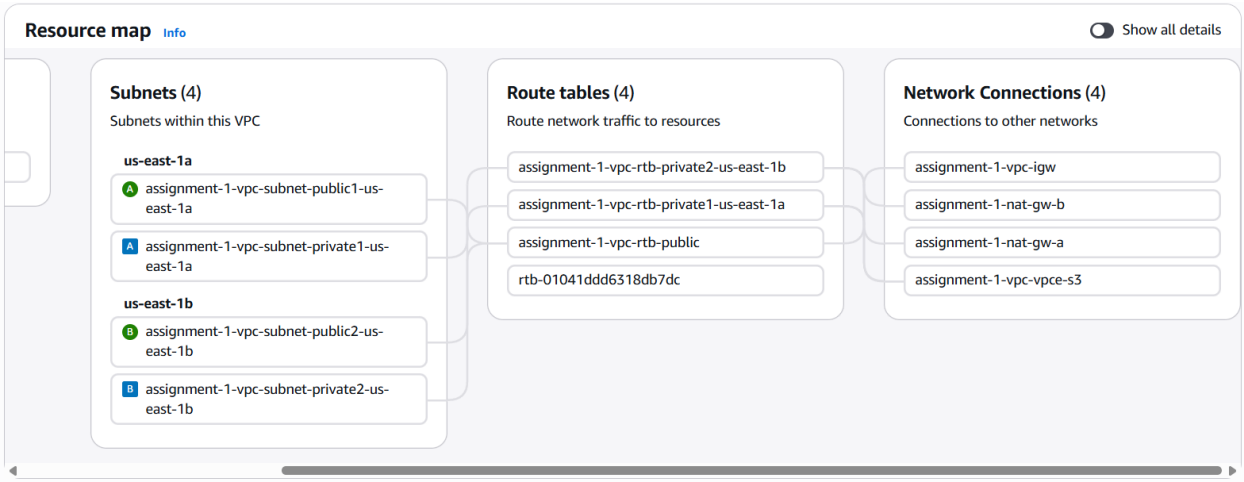
- **ASG:** It creates auto scaled instances in the private subnets.
- **RDS:** A relational database is placed in one of the private subnets which can be accessed by the instances in both private subnets.
- **Auto scaled Instances:** Scales in or out depending on the scaling policy. Also, sends usage metrics to cloud watch.
- **CloudWatch:** Shows usage metrics and alarms for instances.

Core Functionalities

2.1 VPC

A VPC was made with two availability zones containing both a public and a private subnet to ensure higher availability.



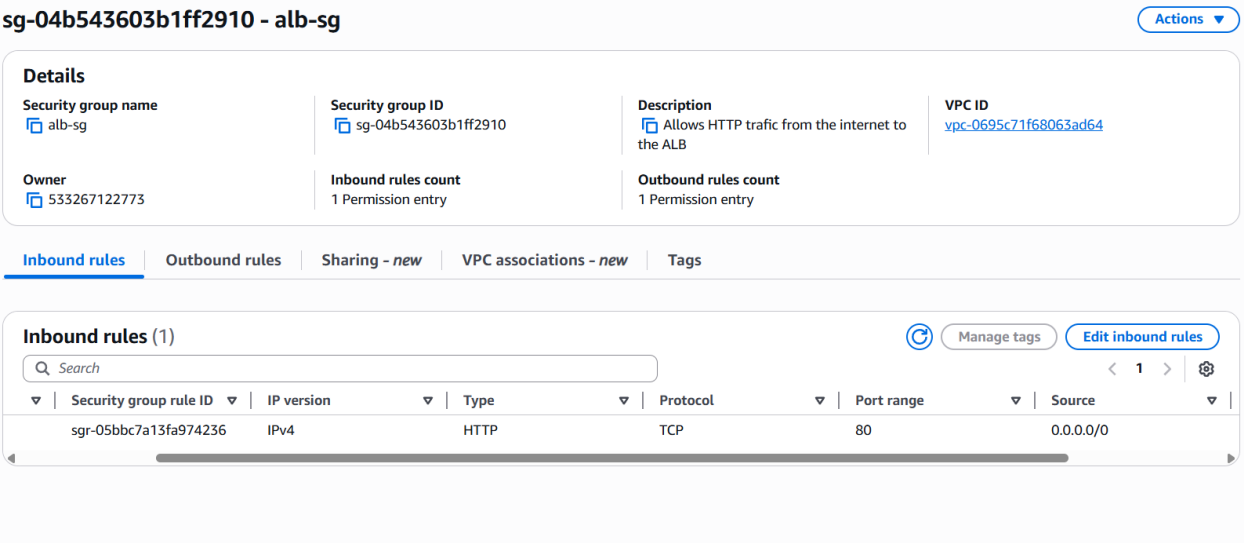


From the routing table we can see that there is an internet gateway attached to the VPC to connect to the internet, 2 NAT gateways in the public subnet to allow private subnet instances to safely go through the NAT to the IGW.

2.2 Security groups

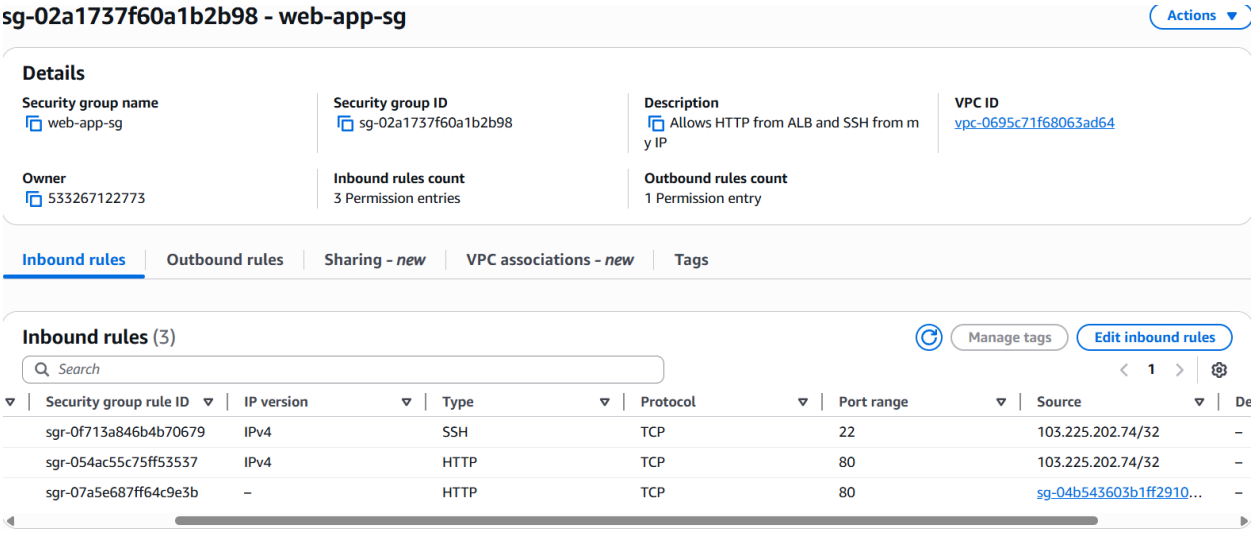
Three security groups were made:

1. alb-sg (for public access to load balancer)



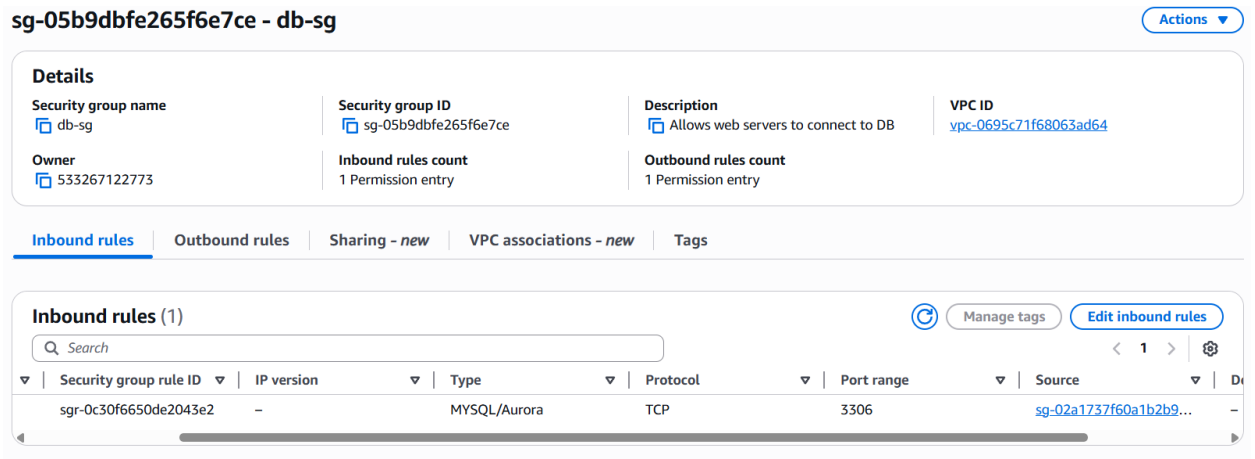
Allows access from any IPv4.

2. web-app-sg (for the web servers in private subnet)



Allows SSH and HTTP traffic from My IP as well as from the Load Balancer.

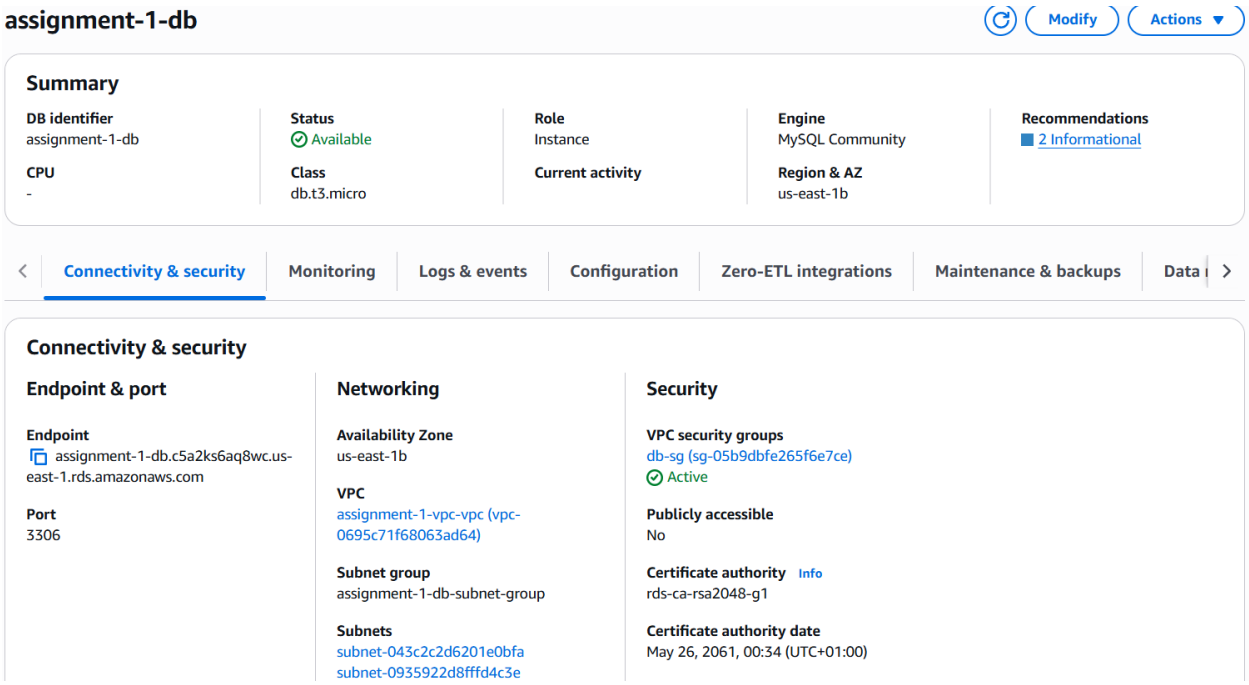
3. db-sg (for the RDS in private subnet)



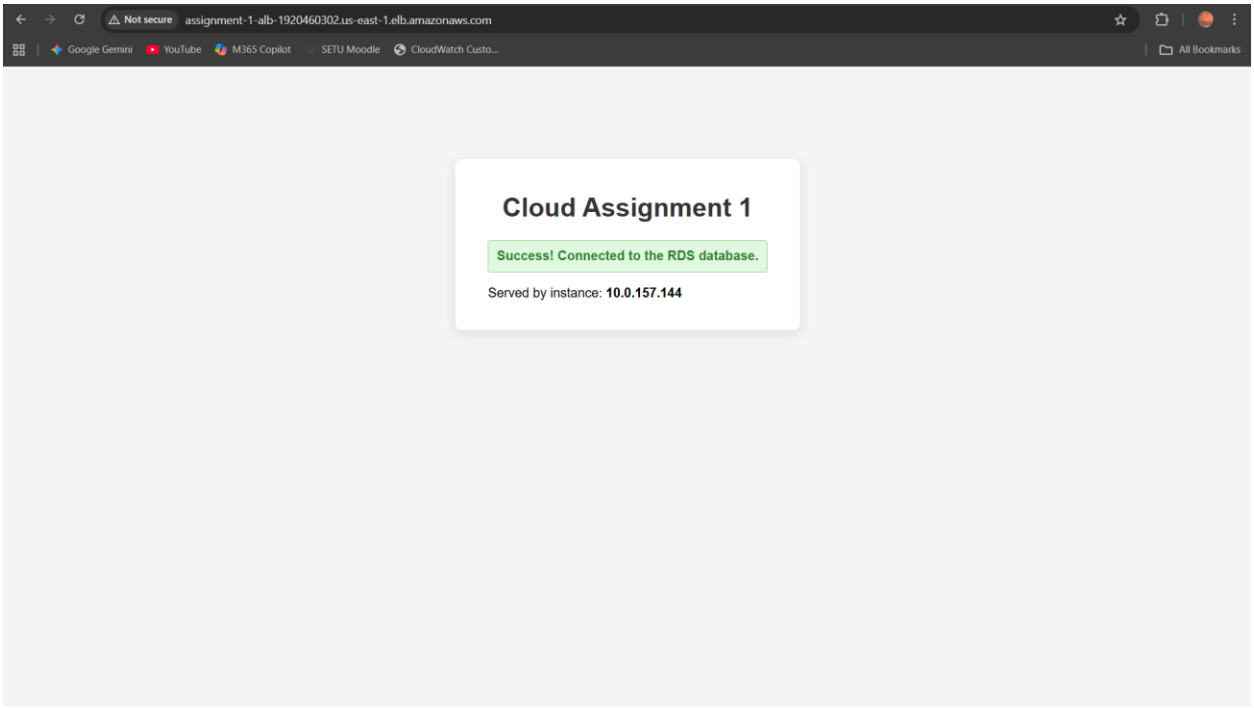
It has the same security rules as in web-app-sg

2.3 RDS Database (Additional Feature)

A MySQL database was created and launched in private subnet in order to ensure it was not publicly accessible.

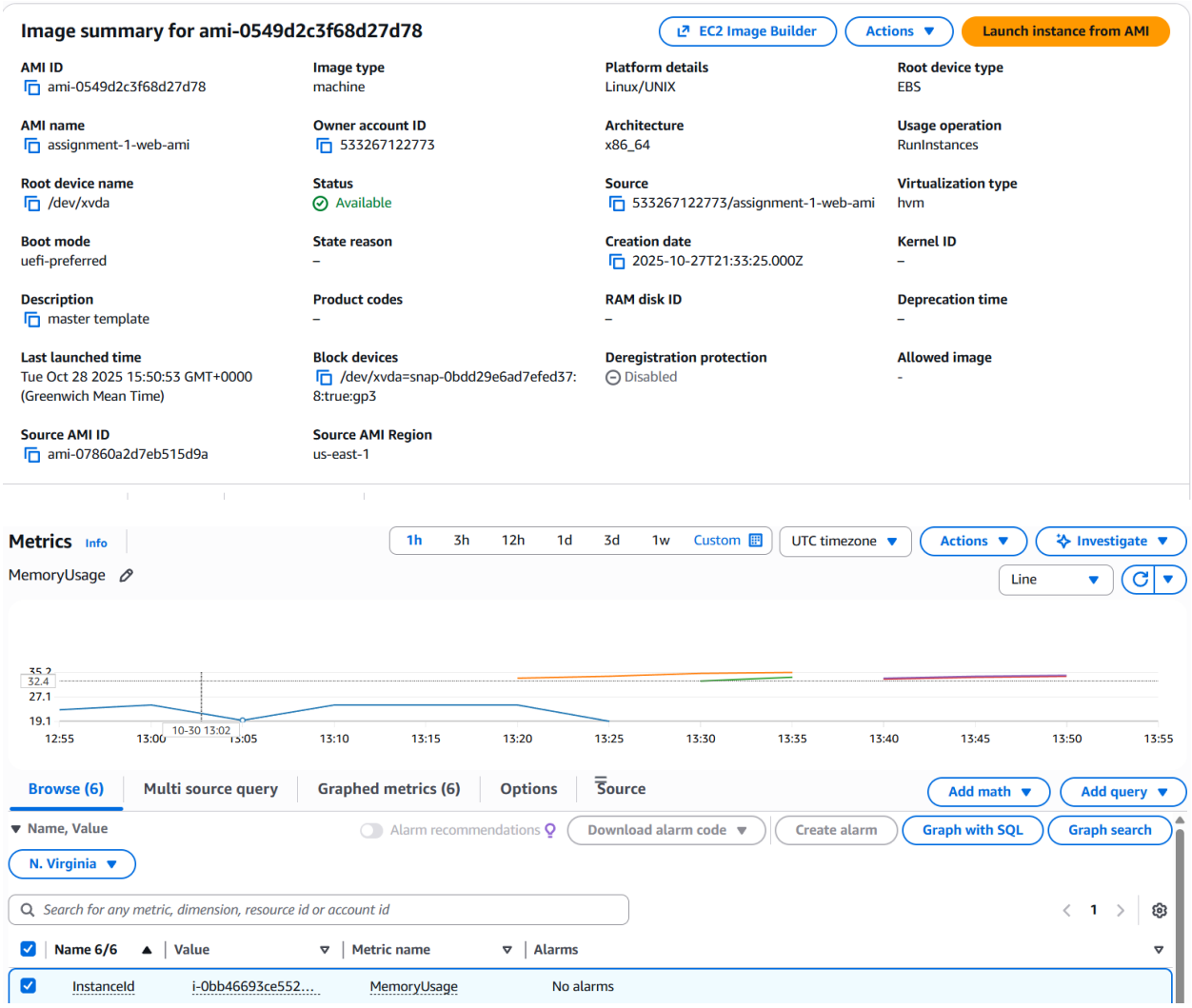


It uses the db-sg security group and allows SSH and HTTP requests from my IP as well as HTTP request from the load balancer.



2.4 Master Instance, Custom CloudWatch metrics & Custom AMI

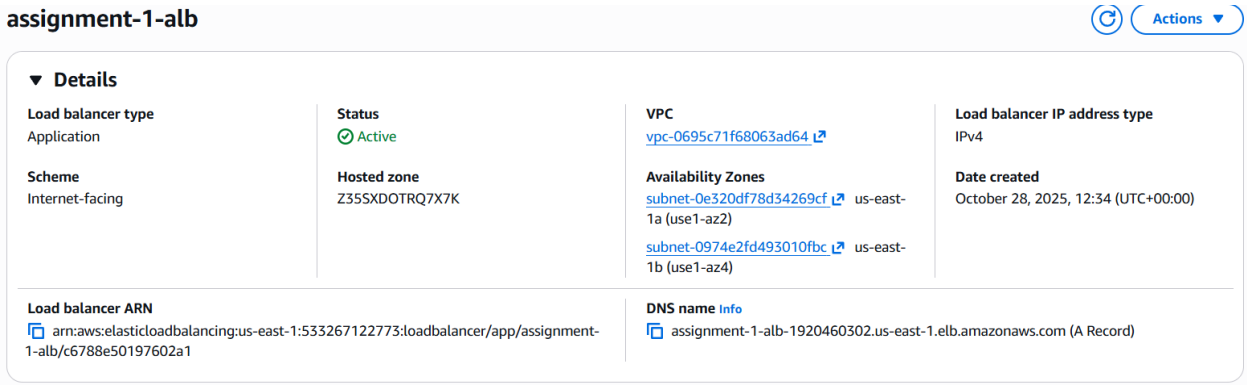
A temporary master instance was created and deployed to public subnet where various packages were installed (e.g. Apache, PHP, MySQL, Cronie, AWS CLI). Index page was created for the purpose of having a basic front end where connection to database as well as instance ID could be viewed. Furthermore, a custom cloud watch metric which calculates ram usage percentage was also pushed. Finally, AMI was generated from this instance to keep a template of the master instance and later the master instance was deleted.



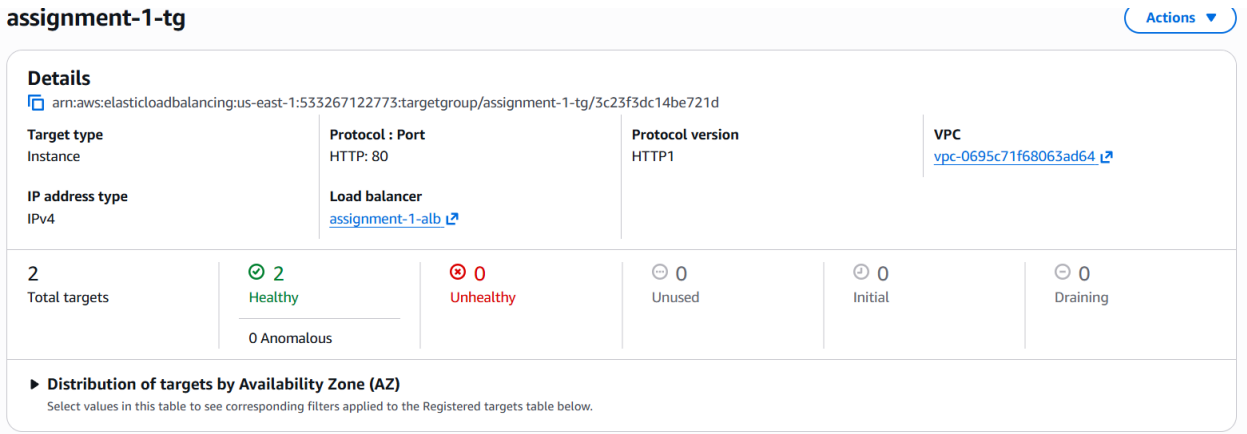
The reason for building custom metrics on ram usage is that, while CPU usage may remain moderate in various scenarios, the ram usage may spike and if remain unnoticed it may cause lags or crashes. So, monitoring ram usage is also equally important.

2.5 ALB & Target Group

An application load balancer was created and launched in the public subnets to distribute incoming traffic from IGW to auto scaled ec2 instances in the target group efficiently as well as any requests to the RDS.



alb-sg security group was used to allow HTTP traffic from the internet (any IPv4).



Curl test showing the load balancer in action

```
C:\Users\GIGABYTE>curl http://assignment-1-alb-1920460302.us-east-1.elb.amazonaws.com
<!DOCTYPE html>
<html>
<head>
    <title>Cloud Assignment 1</title>
    <style>
        body { font-family: Arial, sans-serif; display: grid; place-items: center; min-height: 80vh; background-color: #f4f4f4; }
        .container { background: #fff; border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); padding: 20px 40px; }
        h1 { text-align: center; color: #333; }
        .status { padding: 10px; border-radius: 4px; font-weight: bold; text-align: center; }
        .success { background-color: #e0f8e0; border: 1px solid #a0d0a0; color: #308030; }
        .fail { background-color: #f8e0e0; border: 1px solid #d0a0a0; color: #803030; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Cloud Assignment 1</h1>
        <div class="status success">Success! Connected to the RDS database.</div><p>Served by instance: <strong>10.0.128.96</strong></p>    </div>
</body>
</html>

C:\Users\GIGABYTE>curl http://assignment-1-alb-1920460302.us-east-1.elb.amazonaws.com
<!DOCTYPE html>
<html>
<head>
    <title>Cloud Assignment 1</title>
    <style>
        body { font-family: Arial, sans-serif; display: grid; place-items: center; min-height: 80vh; background-color: #f4f4f4; }
        .container { background: #fff; border-radius: 8px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); padding: 20px 40px; }
        h1 { text-align: center; color: #333; }
        .status { padding: 10px; border-radius: 4px; font-weight: bold; text-align: center; }
        .success { background-color: #e0f8e0; border: 1px solid #a0d0a0; color: #308030; }
        .fail { background-color: #f8e0e0; border: 1px solid #d0a0a0; color: #803030; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Cloud Assignment 1</h1>
        <div class="status success">Success! Connected to the RDS database.</div><p>Served by instance: <strong>10.0.157.144</strong></p>    </div>
</body>
</html>
```

2.6 Auto Scaling (Additional Feature IAC)

I have used Terraform to automate the autoscaling process. The process required:

1. **Creation of launch template**

```
resource "aws_launch_template" "web_lt" {
  name_prefix   = "web-app-lt-"
  image_id      = data.aws_ami.web_ami.id
  instance_type = "t2.micro"
  key_name      = "testKey"

  iam_instance_profile {
    arn = data.aws_iam_instance_profile.cw_role.arn
  }

  network_interfaces {
    associate_public_ip_address = false
    security_groups             = [data.aws_security_group.web_app_sg.id]
  }

  depends_on = [data.aws_iam_instance_profile.cw_role]

  tags = {
    Name = "asg-web-server"
  }
}
```

2. Creation of Autoscaling group

```
resource "aws_autoscaling_group" "web_asg" {
  name                      = "assignment-1-web-asg"
  max_size                  = 4
  min_size                  = 2
  desired_capacity          = 2
  vpc_zone_identifier       = data.aws_subnets.private.ids

  target_group_arns         = [data.aws_lb_target_group.web_tg.arn]

  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }

  tag {
    key                 = "Name"
    value               = "asg-member"
    propagate_at_launch = true
  }

  health_check_type         = "ELB"
  health_check_grace_period = 300
}
```

3. Creation of Autoscaling policies

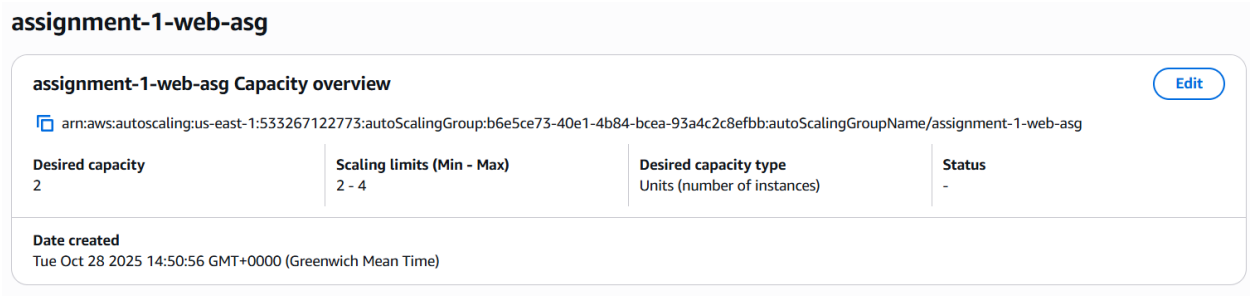
```
resource "aws_autoscaling_policy" "scale_out" {
  name                   = "assignment-1-scale-out-policy"
  autoscaling_group_name = aws_autoscaling_group.web_asg.name
  adjustment_type        = "ChangeInCapacity"
  cooldown               = 300
  scaling_adjustment     = 1

}

resource "aws_autoscaling_policy" "scale_in" {
  name                   = "assignment-1-scale-in-policy"
  autoscaling_group_name = aws_autoscaling_group.web_asg.name
  adjustment_type        = "ChangeInCapacity"
  cooldown               = 300
  scaling_adjustment     = -1

}
```

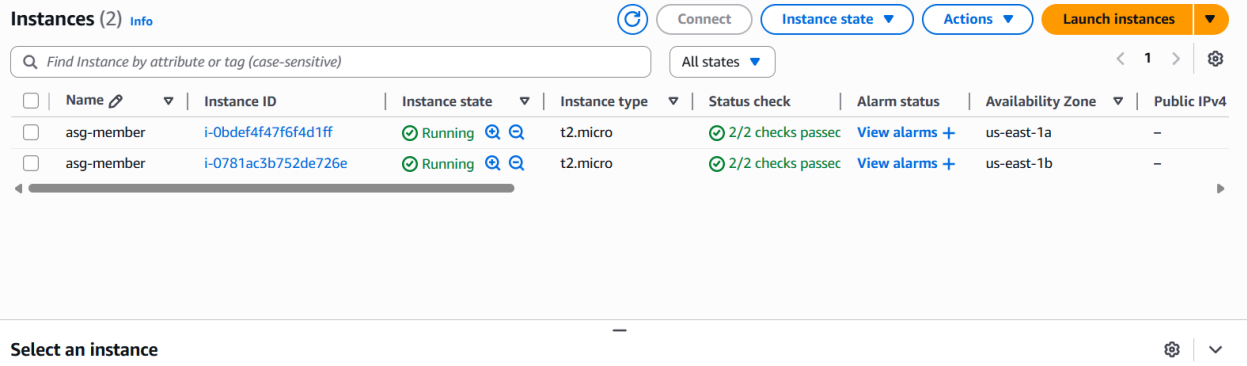
4. Creation of alarms

```hcl
resource "aws_cloudwatch_metric_alarm" "high_memory_alarm" {
  alarm_name          = "asg-high-memory-alarm"
  comparison_operator = "GreaterThanOrEqualToThreshold"
  evaluation_periods  = 2
  metric_name         = "MemoryUsage"
  namespace           = "Assignment1"
  period              = 300
  statistic           = "Average"
  threshold           = 80
  alarm_actions = [aws_autoscaling_policy.scale_out.arn]

  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.web_asg.name
  }


}

resource "aws_cloudwatch_metric_alarm" "low_memory_alarm" {
  alarm_name          = "asg-low-memory-alarm"
  comparison_operator = "LessThanOrEqualToThreshold"
  evaluation_periods  = 2
  metric_name         = "MemoryUsage"
  namespace           = "Assignment1"
  period              = 300
  statistic           = "Average"
  threshold           = 40
  alarm_actions = [aws_autoscaling_policy.scale_in.arn]

  dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.web_asg.name
  }


}
```

I referred to the documentations provided at https://registry.terraform.io/providers/hashicorp/aws/latest/docs to write the code. The terraform folder will be attached with the submission.

Here are some screenshots from AWS console:





Conclusion

This project demonstrated deployment of a secure, scalable and highly available web application on AWS. All core specifications were met and additionally I have included RDS database and automation of auto scaling features using terraform.

The security aspect could be further enhanced for the project by generating SSL/TLS certificate and enabling HTTPS listener on the application load balancer so that all data are encrypted during transit between user and the application.

Plagiarism Declaration

I declare that the work I have presented in this assignment is entirely my own and it has not been copied from any other source, except where I acknowledged using external sources as guide.